BSc (Hons) in **Information Technology Specialized in AI**

**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 04

**Module Objective:** In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

## Part 1: The Logic Engine Architecture

### Step 1.1: Building the Knowledge Base (KB)

Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .
2. Implement a `KnowledgeBase` class based on the following structural requirements:

```
CLASS KnowledgeBase:
   // Attributes
   DECLARE facts AS Set                 // To store unique string facts (e.g., "TargetVisible")
   DECLARE rules AS List                // To store rules as Tuples: ( [list_of_premises], "conclusion_string" )

   // Methods
   FUNCTION tell_fact(fact_string):
       ADD fact_string TO facts

   FUNCTION tell_rule(premise_list, conclusion_string):
       APPEND Tuple(premise_list, conclusion_string) TO rules

   FUNCTION clear_facts():
       EMPTY the facts Set
```

## Part 2: Implementing Forward Chaining

### Step 2.1: The Inference Algorithm

The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward_chain():
   DECLARE new_facts_added = TRUE

   WHILE new_facts_added == TRUE:
       new_facts_added = FALSE

       FOR EACH (premises, conclusion) IN rules:
           IF conclusion IS NOT IN facts:

               // Modus Ponens Check
               IF ALL premises ARE IN facts:
                   ADD conclusion TO facts
                   new_facts_added = TRUE
```

## Part 3: Hooking Logic into the Grid Game

### Step 3.1: Defining the Game Constraints

> Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In your main `agent.py` , instantiate the KB in the `__init__` method: `self.kb = KnowledgeBase()` .
2. Use your `tell_rule` method to define the following safety rules:
   *Rule 1:* `TargetVisible ∧ HasDust ⇒ SafeToEngage`
   *Rule 2:* `SafeToEngage ∧ BloodseekerMissing ⇒ Retreat`

### Step 3.2: Validating Feasibility in A*

> Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
2. Before adding a neighbor to the `open_list` , clear the KB facts.
3. Feed the current percepts for that specific tile into the KB using `tell_fact()` .
4. Run `self.kb.forward_chain()` .
5. If `'Retreat'` is deduced and exists in `self.kb.facts` , mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

## Part 4: Self-Evaluation Test Cases

### 4.1 Automated Validation

Create a file named `test_logic.py` . Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```
from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce SafeToEngage"
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT deduce Retreat"

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.tell_fact('BloodseekerMissing')
    kb.forward_chain()
    assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat to override search"

    print("✅ All Logic Engine Test Cases Passed!")

if __name__ == "__main__":
    test_forward_chaining()
```

## Part 5: Theory-to-Implementation Mapping

> Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

**1. Reachability vs. Feasibility (Apply)**

**In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.**

Reachability is checked in get_neighbors() — it just tests if the cell is inside the grid and not a wall. Feasibility is checked separately in is_tile_feasible() and it builds facts about the tile, runs forward_chain(), and rejects the tile if 'Retreat' gets deduced. So a tile can pass the wall check but still get blocked by the KB.

**2. Declarative vs. Procedural Paradigms (Analyze)**

**Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust' in percepts:` directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?**

Writing if 'TargetVisible' in percepts and 'HasDust' in percepts: hardcodes one rule directly into the agent's logic. The KnowledgeBase instead stores rules as data (tell_rule()) separate from the engine that runs them (forward_chain()), so adding a rule never touches the inference code. If we added 50 new rules, the procedural version would need 50 new nested if statements and get messy fast. the declarative version just gets 50 more tell_rule() calls and the forward_chain() loop stays exactly the same.

**3. Modus Ponens & Horn Clauses (Understand)**

**The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.**

Modus Ponens says: if P → Q and P is true, then Q is true. In forward_chain(), the check if all is testing "is P true?", and self.facts.add(conclusion) is asserting Q once it is and that's one Modus Ponens step. The while loop just repeats this over all rules until nothing new is added, so it chains Modus Ponens multiple times to derive facts that depend on other derived facts.

Finalize and Lock Lab 05 Implementation

**FACULTY OF COMPUTING**